

Day 10: Advanced RAG Patterns

1. 5-line beginner summary

Basic RAG means: search documents, fetch chunks, send them to an LLM, and generate an answer. But basic RAG can fail if chunks are bad, search misses the right content, or the LLM gets incomplete context. Advanced RAG improves retrieval using hybrid search, reranking, query rewriting, and multi-query retrieval. It improves answer quality using grounding, contextual compression, and better prompts. Enterprise RAG is not only about “using a vector database”; it is about retrieving the right evidence and forcing the LLM to answer from that evidence.

2. Descriptive notes

1. Why basic RAG may fail

Basic RAG usually follows this flow:



This looks simple, but real enterprise documents are messy.

Basic RAG may fail because:

Problem	What happens
Bad chunking	Important information is split incorrectly
Missing context	Retrieved chunk does not contain full meaning
Wrong retrieval	Vector search brings similar but not correct chunks
No reranking	Top chunks are not sorted by true relevance
Long documents	Important answer may be spread across many pages
Weak prompt	LLM answers beyond the given context
Hallucination	LLM fills gaps using its own knowledge

Example:

User asks:

What is the notice period for Band 8 employees in India?

Basic RAG may retrieve a chunk about:

Notice period for contractors is 15 days.

But the correct answer may be in another section:

For full-time Band 8 employees in India, notice period is 90 days.

The query and wrong chunk are semantically similar, so vector search may retrieve the wrong one.

2. Poor chunking problem

Chunking means splitting large documents into smaller pieces.

Example document:

Section 4: Employee Exit Policy

Employees in India must serve a notice period based on band level.
Band 6 and Band 7 employees must serve 60 days.
Band 8 and above employees must serve 90 days.
Contractors must give 15 days' notice.

Bad chunking may split it like this:

Chunk 1:
Employees in India must serve a notice period based on band level.
Band 6 and Band 7 employees must serve 60 days.

Chunk 2:
Band 8 and above employees must serve 90 days.
Contractors must give 15 days' notice.

If the user asks:

What is the notice period for Indian Band 8 employees?

Chunk 2 has the answer, but it does not clearly mention “India” or “notice period based on band level”. The missing context may confuse the LLM.

Better chunking:

Chunk:
Section 4: Employee Exit Policy
Employees in India must serve a notice period based on band level.
Band 8 and above employees must serve 90 days.

Good chunks should preserve meaning.

Good chunking principles

Rule	Meaning
Keep related information together	Do not split policy rule from condition
Preserve headings	Section titles give meaning
Use overlap carefully	Some repeated text helps avoid context loss
Chunk by structure	Prefer headings, paragraphs, tables, sections
Avoid very tiny chunks	They lose context
Avoid very large chunks	They add noise

3. Missing context problem

Sometimes the retrieved chunk contains the answer but not enough supporting context.

Example:

Chunk:
The limit is 5 business days.

This chunk is useless alone.

Better chunk:

Travel Reimbursement Policy:
Employees must submit domestic travel reimbursement claims within 5 business days after trip completion.

Now the answer is grounded.

Missing context happens when:

- chunk is too small
- headings are removed
- table headers are separated from table rows
- acronyms are not expanded
- previous paragraph has the condition
- next paragraph has the exception

In enterprise RAG, this is very common because documents contain:

- policies
- contracts
- tables
- process documents
- FAQs
- legal terms
- country-specific rules
- role-specific exceptions

4. Keyword search vs vector search

Keyword search

Keyword search looks for exact words.

Example query:

```
laptop reimbursement policy
```

Keyword search finds documents containing:

```
laptop  
reimbursement  
policy
```

Good for:

- exact product names
- employee IDs
- policy IDs
- legal terms
- error codes
- invoice numbers
- acronyms

Example:

```
POL-IT-447  
ERR_CONN_TIMEOUT  
Form 16  
Band 8
```

Keyword search is very good for these.

Problem:

If user asks:

```
Can I claim money for a work computer?
```

Keyword search may miss the document because the document says:

```
laptop reimbursement
```

The words are different.

Vector search

Vector search finds semantic meaning.

Example:

```
User query:  
Can I claim money for a work computer?
```

Document:
Laptop reimbursement policy

Vector search understands that:

work computer $\approx$ laptop
claim money $\approx$ reimbursement

Good for:

- natural language questions
- synonyms
- meaning-based search
- vague user queries

Problem:

Vector search may retrieve semantically similar but incorrect content.

Example:

Question:
What is reimbursement limit for laptop?

Wrong retrieved chunk:
Mobile phone reimbursement limit is ₹20,000.

This is similar, but not correct.

5. Hybrid search

Hybrid search combines keyword search and vector search.

Hybrid Search = Keyword Search + Vector Search

Why?

Because enterprise queries often contain both meaning and exact terms.

Example:

What is the Band 8 laptop reimbursement limit under POL-IT-447?

This query has:

Meaning:
laptop reimbursement limit

Exact terms:
Band 8
POL-IT-447

Vector search helps understand meaning. Keyword search helps preserve exact matching.

Hybrid search flow

```
User question
↓
Run keyword search
↓
Run vector search
↓
Merge results
↓
Remove duplicates
↓
Score combined results
↓
Send to reranker
```

Simple example

Question:

What is the reimbursement limit for Band 8 employees?

Keyword search may find:

Chunk A: Band 8 employees are eligible for premium device reimbursement.
Chunk B: Band 8 employees must complete approval workflow.

Vector search may find:

Chunk C: Laptop reimbursement limit is ₹80,000 for senior employees.
Chunk D: Device reimbursement applies to full-time employees.

Hybrid search combines these, and reranking chooses the best chunks.

6. Reranking

Reranking means taking retrieved chunks and sorting them again using a smarter relevance model.

Basic retrieval may return:

Top 5 chunks from vector DB:

1. Mobile reimbursement policy
2. Laptop reimbursement overview
3. Band 8 approval workflow
4. Laptop reimbursement limit for Band 8
5. Travel reimbursement policy

The correct chunk is rank 4.

A reranker can move it to rank 1:

After reranking:

1. Laptop reimbursement limit for Band 8

2. Laptop reimbursement overview
3. Band 8 approval workflow
4. Mobile reimbursement policy
5. Travel reimbursement policy

Why reranking is useful

Vector databases are optimized for fast retrieval. Rerankers are optimized for deeper relevance checking.

Think of it like this:

```
Retriever = fast shortlist maker  
Reranker = careful judge
```

Common reranking pattern

```
Retrieve top 20 chunks  
  ↓  
Rerank top 20  
  ↓  
Keep best 5  
  ↓  
Send best 5 to LLM
```

This is better than directly sending top 5 from vector search.

7. Query rewriting

Users do not always ask clear questions.

Example user query:

```
Can I get it?
```

This is unclear.

A query rewriting step may convert it into:

```
Can an employee get laptop reimbursement according to the device reimbursement  
policy?
```

Another example:

```
User query:  
What about India notice?
```

```
Rewritten query:  
What is the employee notice period policy for India?
```

Query rewriting improves retrieval by making the user question more searchable.

Query rewriting is useful when:

- user query is short
- query has pronouns like it, this, that
- query uses vague language
- previous chat history is needed
- query needs business-specific terms

Example with chat history

Conversation:

User: Tell me about laptop reimbursement.
Assistant: It depends on employee band and country.
User: What about Band 8 in India?

The current question alone is incomplete.

Rewritten standalone query:

What is the laptop reimbursement policy for Band 8 employees in India?

This improves retrieval.

8. Multi-query retrieval

Sometimes one query is not enough.

Multi-query retrieval means generating multiple versions of the question and searching with all of them.

Original query:

Can I claim work-from-home setup cost?

Generated queries:

1. Work from home equipment reimbursement policy
2. Home office setup claim eligibility
3. Employee reimbursement for WFH desk and chair
4. Remote work allowance policy

Then RAG searches for all these queries and merges the results.

Why multi-query retrieval helps

Different documents may use different words:

User says	Document may say
work-from-home setup	remote work equipment
claim	reimbursement
laptop money	device allowance
leave	time off

User says	Document may say
firing	termination

Multi-query retrieval increases the chance of finding the correct document.

Risk

It can also bring more noise.

So usually we combine it with:

- deduplication
- reranking
- metadata filters
- score thresholds

9. Contextual compression

Contextual compression means removing irrelevant parts from retrieved chunks before sending them to the LLM.

Example retrieved chunk:

Laptop Reimbursement Policy

This policy applies to employees in India, USA, UK, and Canada.
 Employees must submit invoices within 30 days.
 Band 6 employees are eligible for ₹50,000.
 Band 7 employees are eligible for ₹65,000.
 Band 8 employees are eligible for ₹80,000.
 Contractors are not eligible.
 Finance team reviews all claims.

User asks:

What is the laptop reimbursement limit for Band 8 in India?

Compressed context:

Laptop Reimbursement Policy:
 For employees in India, Band 8 employees are eligible for ₹80,000 laptop reimbursement.

This reduces noise and saves tokens.

Why contextual compression is important

LLMs have limited context windows. Even when the context window is large, more text does not always mean better answer.

Too much irrelevant context can cause:

- confusion
- higher cost

- slower response
- wrong answer
- hallucination

Contextual compression keeps only answer-relevant evidence.

10. Context grounding

Context grounding means forcing the LLM to answer only using retrieved evidence.

A grounded answer should say:

According to the retrieved policy, Band 8 employees in India are eligible for ₹80,000 laptop reimbursement.

It should not say:

Usually companies provide around ₹1,00,000.

Because that is not from the given context.

Grounding rules

A good RAG system should tell the LLM:

- Use only the provided context.
- Do not use outside knowledge.
- If answer is not found, say you do not have enough information.
- Cite the source document or section.
- Do not guess.

Example grounded answer

Context:

Source: Device Reimbursement Policy, Section 3.2
Band 8 employees in India are eligible for laptop reimbursement up to ₹80,000.

Question:

What is the laptop reimbursement limit for Band 8 employees in India?

Answer:

Band 8 employees in India are eligible for laptop reimbursement up to ₹80,000, according to Device Reimbursement Policy, Section 3.2.

11. Prompt optimization for RAG

A weak prompt:

Answer the user's question using the context.

Better RAG prompt:

You are an enterprise policy assistant.

Answer the user's question using only the provided context.

Rules:

1. Do not use outside knowledge.
2. If the answer is not present, say: "I could not find this in the provided documents."
3. Mention the source document name and section when available.
4. Keep the answer concise.
5. If there are conditions or exceptions, include them.

Why prompt optimization matters

Even with good retrieval, a weak prompt can produce poor answers.

Good prompt design helps with:

- reducing hallucination
- improving structure
- enforcing citations
- handling missing answers
- explaining conditions and exceptions

Enterprise RAG prompt should include

Prompt part	Purpose
Role	Defines assistant behavior
Context	Retrieved evidence
User question	Actual question
Rules	Prevent hallucination
Output format	Makes answer consistent
Citation instruction	Improves traceability

12. Handling long documents

Enterprise documents can be very long:

- HR policy manuals
- legal contracts
- technical architecture documents
- compliance manuals
- audit reports
- product documentation

Long documents are difficult because the answer may be spread across many sections.

Techniques for long documents

A. Structure-aware chunking

Split by:

- headings
- sections
- subsections
- tables
- clauses
- paragraphs

Instead of blindly splitting every 500 tokens.

B. Parent-child retrieval

Store small chunks for search, but return larger parent sections for context.

Example:

```
Small child chunk:
Band 8 employees are eligible for ₹80,000.

Parent section:
Device Reimbursement Policy
Section 3: Eligibility and Limits
Full explanation with country, band, exception, approval workflow.
```

Search uses the small chunk. LLM receives the parent section.

This solves missing context.

C. Hierarchical retrieval

Retrieve in levels:

```
Level 1: Find relevant document
Level 2: Find relevant section
Level 3: Find relevant paragraph or table row
```

This is useful for very large document collections.

D. Summary indexing

Create summaries for long documents and store them separately.

Search first over summaries, then drill into detailed chunks.

Example:

```
Document summary:
This policy explains laptop reimbursement eligibility, limits, approval process,
and country-specific rules.
```

Then retrieve relevant detailed sections.

E. Table-aware retrieval

Tables need special care.

Bad chunk:

Band 8 India ₹80,000

Better chunk:

Device Reimbursement Table: Country = India Employee Band = Band 8 Laptop reimbursement limit = ₹80,000 Approval required = Manager + Finance

Table headers must be preserved.

13. Reducing hallucination

Hallucination means the LLM gives an answer that sounds correct but is not supported by evidence.

In RAG, hallucination can happen when:

- | |
|--|
| <ul style="list-style-type: none">- retrieved context is wrong- context is incomplete- prompt allows guessing- user asks something not in documents- LLM uses general knowledge- citations are not enforced |
|--|

Ways to reduce hallucination

Method	How it helps
Better chunking	Gives complete meaning
Hybrid search	Improves retrieval accuracy
Reranking	Puts best evidence first
Metadata filtering	Reduces wrong document matches
Grounding prompt	Prevents guessing
Citation requirement	Forces source-backed answers
Answerability check	Detects when answer is not available
Context compression	Removes distracting text
Evaluation	Finds failure cases

Example

Question:

Are contractors eligible for laptop reimbursement?

Context:

The document discusses full-time employees only.

Bad answer:

Contractors are probably not eligible.

Good grounded answer:

I could not find contractor eligibility in the provided context. The retrieved policy only mentions full-time employees.

14. Improving answer quality

Answer quality depends on both retrieval quality and generation quality.

Good RAG answer = Right context + Good reasoning + Grounded response

Ways to improve answer quality

A. Improve ingestion

Clean the documents before indexing.

- remove duplicate pages
- preserve headings
- preserve tables
- extract metadata
- remove noisy footer/header text

B. Improve chunking

Use structure-aware chunks.

- by section
- by paragraph
- by table
- with heading context
- with controlled overlap

C. Improve retrieval

Use:

- hybrid search
- metadata filtering

- multi-query retrieval
- query rewriting
- top-k tuning

D. Improve ranking

Use reranking before sending context to LLM.

Retrieve top 30
Rerank
Send top 5

E. Improve prompt

Force the model to:

- answer only from context
- cite sources
- say when not enough information is available
- include conditions and exceptions

F. Improve evaluation

Test RAG with real questions.

Track:

- retrieval accuracy
- answer correctness
- citation correctness
- hallucination rate
- latency
- cost

3. Easy examples

Example 1: HR policy assistant

User asks:

What is the notice period for Band 8 employees in India?

Basic RAG may retrieve:

Contractors must give 15 days notice.

Advanced RAG flow:

1. Rewrite query:
"Notice period policy for Band 8 full-time employees in India"

2. Hybrid search:
Keyword search for "Band 8", "India", "notice period"
Vector search for semantic meaning
3. Rerank:
Select chunk with exact Band 8 + India + notice period
4. Grounded answer:
"Band 8 employees in India must serve 90 days notice, according to HR Exit Policy Section 4."

Example 2: IT support RAG

User asks:

How do I fix VPN error 809?

Keyword search is important because:

809

is an exact error code.

Vector search may understand:

VPN connection issue

Hybrid search combines both.

Good answer:

VPN error 809 is usually resolved by checking the VPN gateway, firewall ports, and network profile. According to IT Troubleshooting Guide Section 7.3, first restart the VPN client, then verify UDP port access, then raise a ticket if the error continues.

Example 3: Legal contract RAG

User asks:

Can the vendor terminate the agreement without notice?

The answer may be spread across clauses:

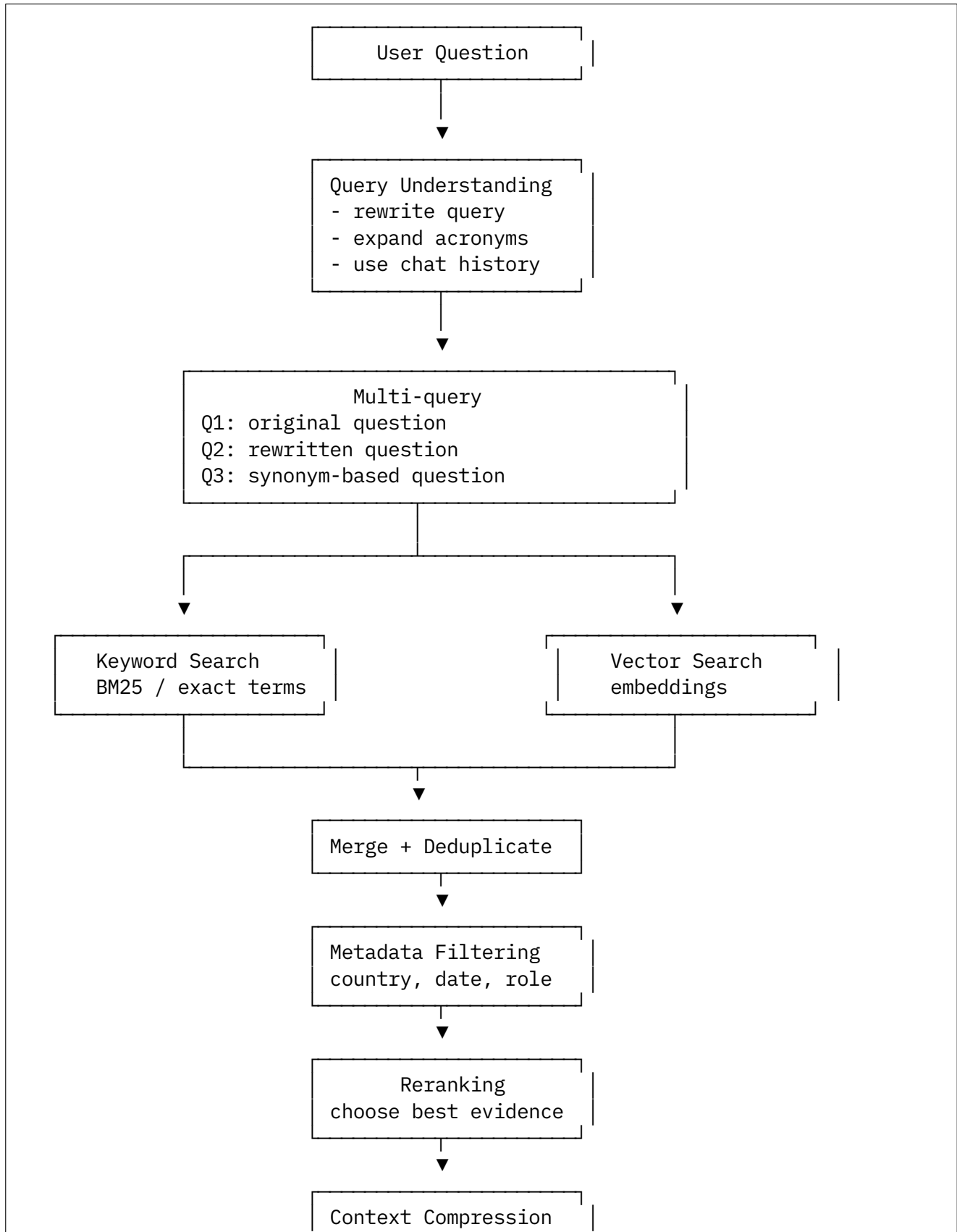
Clause 8.1: Termination for convenience requires 30 days notice.
Clause 8.2: Termination for breach requires written notice and cure period.
Clause 8.3: Immediate termination is allowed only for fraud or regulatory violation.

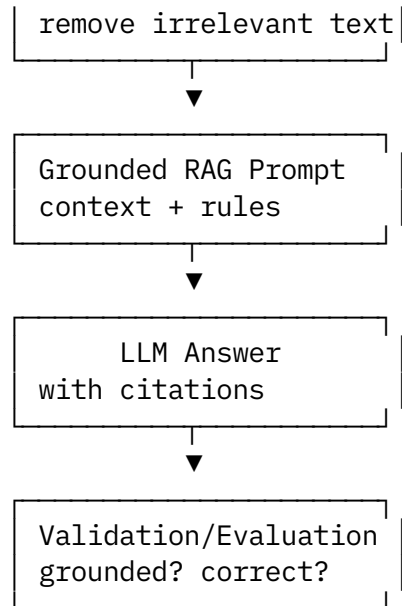
Advanced RAG should retrieve all related clauses, not only one chunk.

Good answer:

The vendor generally cannot terminate without notice, except in specific cases such as fraud or regulatory violation. The contract requires 30 days notice for termination for convenience and written notice with cure period for breach.

4. ASCII diagram showing advanced RAG





5. Pseudocode for hybrid search plus reranking

```
FUNCTION hybrid_search_with_reranking(user_question, filters, top_k):
```

```
    # Step 1: Rewrite user query
    rewritten_query = rewrite_query(user_question)

    # Step 2: Generate multiple query versions
    query_list = generate_multi_queries(rewritten_query)

    all_results = EMPTY_LIST

    FOR each query IN query_list:

        # Step 3: Run keyword search
        keyword_results = keyword_search(
            query = query,
            filters = filters,
            limit = 20
        )

        # Step 4: Run vector search
        query_embedding = embedding_model.embed(query)

        vector_results = vector_search(
            embedding = query_embedding,
            filters = filters,
            limit = 20
        )

        # Step 5: Add both result types
        all_results.ADD(keyword_results)
        all_results.ADD(vector_results)
```

```

# Step 6: Remove duplicate chunks
unique_results = deduplicate_by_chunk_id(all_results)

# Step 7: Combine scores
scored_results = combine_keyword_and_vector_scores(unique_results)

# Step 8: Keep a candidate shortlist
candidate_chunks = select_top_n(scored_results, n = 30)

# Step 9: Rerank using cross-encoder or reranking model
reranked_chunks = reranker.rank(
    question = user_question,
    chunks = candidate_chunks
)

# Step 10: Select final top chunks
final_chunks = select_top_n(reranked_chunks, n = top_k)

RETURN final_chunks

```

6. Pseudocode for grounded answer generation

```

FUNCTION generate_grounded_answer(user_question, retrieved_chunks):

    # Step 1: Check if enough context exists
    IF retrieved_chunks is EMPTY:
        RETURN "I could not find relevant information in the provided
documents."

    # Step 2: Compress context
    compressed_context = contextual_compressor.compress(
        question = user_question,
        chunks = retrieved_chunks
    )

    # Step 3: Build grounded prompt
    prompt = ""
    You are an enterprise RAG assistant.

    Answer the user's question using only the provided context.

    Rules:
    1. Do not use outside knowledge.
    2. If the answer is not found in the context, say:
        "I could not find this information in the provided documents."
    3. Include source names or section names when available.
    4. Mention important conditions, exceptions, and limits.
    5. Do not guess.

    Context:
    {compressed_context}

```

```
User question:
{user_question}

Answer:
"""

# Step 4: Ask LLM to answer
answer = llm.generate(prompt)

# Step 5: Validate answer grounding
grounding_score = check_answer_supported_by_context(
    answer = answer,
    context = compressed_context
)

IF grounding_score is LOW:
    RETURN "I could not confidently answer this from the provided
documents."

# Step 6: Return grounded answer
RETURN answer
```

7. Common mistakes

Mistake 1: Thinking vector search alone is enough

Vector search is powerful, but enterprise data often needs exact matching.

Example:

```
Policy ID: POL-HR-2026
Error code: VPN-809
Employee band: Band 8
Country: India
```

For these, keyword search is very important.

Mistake 2: Making chunks too small

Bad chunk:

```
The limit is ₹80,000.
```

Better chunk:

```
For Band 8 employees in India, the laptop reimbursement limit is ₹80,000.
```

Small chunks often lose meaning.

Mistake 3: Making chunks too large

Very large chunks may contain too many topics.

Example:

```
Laptop policy + mobile policy + travel policy + leave policy
```

The LLM may get confused and answer from the wrong part.

Mistake 4: Ignoring metadata

Documents usually have useful metadata:

```
country = India  
department = HR  
employee_type = full-time  
policy_version = 2026  
document_type = policy
```

Without metadata filtering, RAG may retrieve correct-looking but wrong-region documents.

Example:

```
India notice period ≠ USA notice period
```

Mistake 5: Not using reranking

Direct vector DB top-k is not always reliable.

Better pattern:

```
Retrieve top 30  
Rerank top 30  
Send best 5 to LLM
```

Mistake 6: Sending too much context to the LLM

More context is not always better.

Too much context can cause:

- higher cost
- slower answer
- more confusion
- lower accuracy

Use contextual compression.

Mistake 7: Allowing the LLM to guess

Bad prompt:

```
Answer using your knowledge.
```

Good prompt:

Answer only from the provided context. If not found, say you could not find it.

Mistake 8: No citation or source tracking

Enterprise users need to trust the answer.

Bad answer:

The limit is ₹80,000.

Better answer:

The limit is ₹80,000, according to Device Reimbursement Policy, Section 3.2.

Mistake 9: Not testing with real user questions

A RAG system may look good in a demo but fail with real queries.

Test with:

- short questions
- vague questions
- multi-part questions
- questions with acronyms
- questions needing exact values
- questions where answer is not present

Mistake 10: Treating RAG as only an LLM problem

RAG is a full system problem.

It includes:

- document ingestion
- parsing
- chunking
- metadata extraction
- embedding
- indexing
- retrieval
- reranking
- prompting
- generation
- evaluation
- monitoring

Final revision sheet

Basic RAG:
Search → Retrieve → Generate

Advanced RAG:

Rewrite query → Multi-query retrieval → Hybrid search → Metadata filter
→ Rerank → Compress context → Grounded prompt → Cited answer

Important ideas:

- Keyword search finds exact terms.
- Vector search finds meaning.
- Hybrid search combines both.
- Reranking improves result order.
- Query rewriting makes unclear questions searchable.
- Multi-query retrieval increases recall.
- Contextual compression removes noise.
- Grounding reduces hallucination.
- Good chunking is critical.
- Enterprise RAG needs evaluation and source traceability.

For interviews, remember this simple sentence:

Advanced RAG improves basic RAG by making retrieval more accurate, context more relevant, and generation more grounded.

Yes. In Advanced RAG, we can use **cross-encoders, small LLMs, embedding models, or rule-based filters** to keep only the most relevant context.

But each is used at a different stage.

Simple answer

For filtering relevant context, we commonly use:

Method	Used for	Example
Embedding model	Fast semantic search	Find top 50 chunks
BM25 / keyword search	Exact word matching	Find chunks with "Band 8", "India"
Cross-encoder reranker	Reorder retrieved chunks by relevance	Move best chunk from rank 8 to rank 1
Small LLM	Compress or extract only useful lines	Remove irrelevant policy text
Rules / metadata filters	Filter by document type, country, date, role	country = India, policy_version = latest
Big LLM	Final grounded answer generation	Generate final answer from selected context

Important difference

Cross-encoder is mostly used for reranking

It checks:

Question + Chunk

together and gives a relevance score.

Example:

Question:

What is the laptop reimbursement limit for Band 8 employees in India?

Chunk A:

Band 8 employees in India are eligible for ₹80,000 laptop reimbursement.

Chunk B:

Band 8 employees must complete annual compliance training.

Chunk C:

Employees in India can claim mobile reimbursement up to ₹20,000.

The cross-encoder may score:

Chunk A: 0.95

Chunk C: 0.62

Chunk B: 0.30

So it keeps Chunk A at the top.

Small LLM is often used for contextual compression

After retrieval and reranking, the chunk may still be long.

Original chunk:

Laptop Reimbursement Policy

This policy applies to employees in India, USA, UK, and Canada.

Employees must submit invoices within 30 days.

Band 6 employees are eligible for ₹50,000.

Band 7 employees are eligible for ₹65,000.

Band 8 employees are eligible for ₹80,000.

Contractors are not eligible.

Finance team reviews all claims.

User asks:

What is the laptop reimbursement limit for Band 8 in India?

A small LLM or compression model extracts only:

For employees in India, Band 8 employees are eligible for ₹80,000 laptop reimbursement.

This is called **contextual compression**.

Typical enterprise flow

```
User question
  ↓
Query rewriting
  ↓
Hybrid search
  ↓
Retrieve top 30-50 chunks
  ↓
Cross-encoder reranker
  ↓
Keep top 5-10 chunks
  ↓
Small LLM / extractor compresses context
  ↓
Big LLM generates grounded answer
```

Which model is used where?

1. Embedding model

Used for vector search.

Purpose:
Find semantically similar chunks quickly.

Example models:

```
text-embedding models
sentence-transformers
bge embeddings
e5 embeddings
```

2. Cross-encoder reranker

Used after initial retrieval.

Purpose:
Carefully compare question and chunk together.

It is more accurate than embedding similarity, but slower.

Example reranker models:

```
bge-reranker
Cohere Rerank
cross-encoder/ms-marco models
ColBERT-style rerankers
```

3. Small LLM / extraction model

Used for contextual compression.

Purpose:
Remove irrelevant sentences and keep only useful evidence.

Example:

Input:
Question + long chunk

Output:
Only lines relevant to the question

4. Big LLM

Used for final answer generation.

Purpose:
Generate a clear answer using only the compressed context.

Example:

GPT / Claude / Gemini / Llama / Granite / Mixtral etc.

In IBM context, you may hear about **IBM Granite models** or **watsonx.ai foundation models** for enterprise GenAI use cases.

Very important interview point

Do not say:

We use cross-encoder to compress the context.

Better answer:

We usually use a cross-encoder for reranking retrieved chunks. For contextual compression, we can use a smaller LLM, extractive model, or rule-based sentence filter to keep only the relevant evidence before sending it to the final LLM.

Simple comparison

Step	Model type	Why
Retrieve candidates	Embedding model + BM25	Fast search
Improve order	Cross-encoder reranker	Better relevance scoring
Remove noise	Small LLM / extractor	Contextual compression
Generate answer	Larger LLM	Fluent grounded response

Practical example

Question:

What is the laptop reimbursement limit for Band 8 employees in India?

Retrieved chunks:

Chunk 1: Laptop policy for India and employee bands.
Chunk 2: Mobile reimbursement policy.
Chunk 3: Band 8 training policy.
Chunk 4: Laptop reimbursement approval workflow.

Cross-encoder reranker selects:

Chunk 1
Chunk 4

Context compressor extracts:

Band 8 employees in India are eligible for ₹80,000 laptop reimbursement.
Manager and Finance approval are required.

Final LLM answers:

Band 8 employees in India are eligible for laptop reimbursement up to ₹80,000.
Approval from Manager and Finance is required.

So yes, we use models to filter relevant context, but usually:

Cross-encoder = rerank relevant chunks
Small LLM/extractor = compress relevant context
Big LLM = generate final grounded answer